



Lambdas e Streams

Aula 14 · Bloco 4 — Java Moderno

Descrever o resultado, não o passo a passo

Nesta aula

1. Interface funcional e lambda
2. Ordenar com `Comparator`
3. Streams — `filter`, `map`, `sorted`
4. `Optional` — dizer "pode não haver" sem `null`
5. Agrupar com `Collectors`

 Nada aqui substitui o que você aprendeu. Mas boa parte dos laços do Bloco 3 vira **uma linha legível**.

Interface funcional

Lembre da aula 08: interface é contrato. Quando ela tem **um único método abstrato**, é uma **interface funcional** — e o Java aceita uma sintaxe enxuta.

```
public interface Filtro {  
    boolean testar(Livro livro);  
}  
  
// o jeito verboso – classe anônima  
Filtro disponiveis = new Filtro() {  
    @Override  
    public boolean testar(Livro livro) { return livro.estaDisponivel(); }  
};  
  
// o mesmo, com lambda  
Filtro disponiveis = livro -> livro.estaDisponivel();
```

Anatomia da lambda

```
() -> System.out.println("oi")           // sem parâmetro
x -> x * 2                                // um parâmetro, retorno implícito
(a, b) -> a + b                           // dois parâmetros
(a, b) -> {                               // corpo com várias linhas: precisa de return
    int soma = a + b;
    return soma * 2;
}
```

E a *method reference*, atalho para quando a lambda só repassa o argumento:

```
lista.forEach(item -> System.out.println(item));
lista.forEach(System.out::println);           // idêntico, mais curto
```

Ordenando com **Comparator**

Ordenar objetos exige responder: **por qual critério?**

```
acervo.sort((a, b) -> a.getTitulo().compareTo(b.getTitulo()));  
acervo.sort(Comparator.comparing(Livro::getTitulo));           // mais legível  
acervo.sort(Comparator.comparingInt(Livro::getAno).reversed()); // recentes 1º  
  
// dois critérios: ano e, no empate, título  
acervo.sort(Comparator.comparingInt(Livro::getAno)  
            .thenComparing(Livro::getTitulo));
```

💡 **compareTo** devolve negativo, zero ou positivo. O **Comparator.comparing** monta isso a partir de um getter — prefira essa forma.



Stream: o quê, não o como

```
// Antes: COMO fazer
List<Livro> d = new ArrayList<>();
for (Livro l : acervo) {
    if (l.estaDisponivel()) d.add(l);
}

// Agora: O QUE você quer
List<Livro> d = acervo.stream()
    .filter(Livro::estaDisponivel)
    .toList();
```

As operações mais usadas

```
acervo.stream().filter(l -> l.getAno() < 1950).toList(); // seleciona
acervo.stream().map(Livro::getTitulo).toList();           // TRANSFORMA

acervo.stream()                                           // ordena e corta
    .sorted(Comparator.comparingInt(Livro::getAno).reversed())
    .limit(3).toList();

acervo.stream().filter(Livro::estaDisponivel).count();   // respostas
acervo.stream().anyMatch(l -> l.getAutor().contains("Machado"));
```

Encadeando

```
List<String> titulosDisponiveis = acervo.stream()  
    .filter(Livro::estaDisponivel)  
    .sorted(Comparator.comparing(Livro::getTitulo))  
    .map(Livro::getTitulo)  
    .toList();
```

Lê-se como uma frase: *"dos disponíveis, ordenados por título, pegue os títulos"*.

⚠ **A stream não altera a lista original** — produz um resultado novo. E é de **uso único**: depois de um `.toList()`, aquela stream acabou.

Números: soma, média, máximo

```
double total = produtos.stream()
    .mapToDouble(Produto::getPreco)    // vira fluxo de double
    .sum();

double media = alunos.stream()
    .mapToDouble(Aluno::calcularMedia)
    .average()
    .orElse(0);                        // e se a lista estiver vazia?
```

Repare no `.orElse(0)`: a média de uma lista vazia **não existe**. O Java te obriga a decidir o que fazer nesse caso — e isso é o `Optional` chegando.

Optional : "pode não haver", sem null

Na aula 09, busca sem resultado devolvia `null` — e quem esquecesse de verificar ganhava um `NullPointerException`.

```
public Optional<Livro> buscarPorCodigo(String codigo) {  
    return acervo.stream()  
        .filter(l -> l.getCodigo().equalsIgnoreCase(codigo))  
        .findFirst();           // devolve Optional<Livro>  
}
```

Uma caixinha que **pode ou não** conter valor — e obriga quem recebe a considerar as duas hipóteses.

Os três desfechos

```
Optional<Livro> resultado = service.buscarPorCodigo("L001");

resultado.ifPresent(System.out::println);           // se houver, faça

Livro livro = resultado.orElseThrow(
    () -> new ItemNaoEncontradoException("L001")); // se não houver, exploda

String titulo = resultado.map(Livro::getTitulo)
    .orElse("(não encontrado)"); // ou um padrão
```

Agrupando com **Collectors**

Relatórios ficam curtos — e o **Map** sai pronto:

```
Map<String, Long> porCategoria = acervo.stream()
    .collect(Collectors.groupingBy(Livro::getCategoria,
        Collectors.counting()));
// {Romance=4, Técnico=7, Infantil=2}

String lista = acervo.stream()
    .map(Livro::getTitulo)
    .collect(Collectors.joining(", "));
```

Quando **não** usar stream

Quando o laço tradicional for mais claro.

Stream de cinco operações aninhadas,
com lambdas de três linhas cada,
é **pior** que um `for` honesto.

O objetivo é **legibilidade**.

Ficou difícil de ler? Volte ao laço.

Exercícios da aula

Na pasta `aula-14/` :

1. **`Ordenacao.java`** — três ordenações com `comparing` e `thenComparing` ;
2. **`Refatorar.java`** — três laços da aula 09 virando stream, com a versão antiga comentada acima;
3. **`Relatorio.java`** — aprovados, média geral, maior média e reprovados em ordem;
4. **`Busca.java`** — troque `null` por `Optional` e mostre os três desfechos;
5. **Desafio  `Estatisticas.java`** — CSV da aula 13 + `groupBy` + relatório em arquivo.

Próxima aula

Aula 15 — Do requisito ao sistema

Aula-laboratório: o seu sistema,
do desenho ao commit final.